Anna Zheng - Reflection 2

In the second phase of the Sensing the World module, we transitioned into computer engineering, focusing on microcontrollers, circuit design, and MicroPython programming. We began by exploring the fundamentals of algorithm planning, constructing block diagrams and flowcharts to map out systems before writing any code. Specifically, we analyzed Dr. Valvano's security system, which utilized contact switches and a flashing LED alarm, and mapped its logic from a functional description to a flowchart. We also learned how to build a portfolio using a GitHub Copilot and our GitHub repositories to document our projects. Furthermore, we created a FYRE-Assignments repository to act as version control, and we configured the Arduino Lab for MicroPython IDE to program our Arduino Nano ESP32 microcontrollers. Afterwards, we wrote our first MicroPython scripts and learned how to utilize a while loop and the machine module to make a built-in LED blink. Over the next classes, we built circuits on solderless breadboards using components like potentiometers, momentary DPST contact switches, photoresistors, and LEDs. We advanced into using Pulse Width Modulation (PWM) to control servo motor angles through square waves, where adjusting the duty cycle dictates the rotational position. Finally, we utilized the microcontroller's Analog-to-Digital Converter (ADC) to collect voltage readings from an LM393 raindrop sensor across varying wetness conditions, exporting the data to CSV files and using Google Sheets to calculate averages and standard deviations, and graph scatter plots with trendlines.

To accomplish these tasks, my partner and I worked closely together. My partner took the lead on hardware assembly, configuring the DC power supply to 5V and mapping the physical pinout connections on our breadboards to ensure the ICs were properly placed across the gaps without causing shorts. Meanwhile, I led the software development, experimenting with ChatGPT to write our MicroPython code. Utilizing AI was really challenging. For instance, our initial prompt for the switch-controlled servo caused the motor arm to automatically snap to a 180º direction without reading the switch input first. We successfully resolved this by revising our prompts to specify the exact pin numbers, assigning our switch to GPIO 17 and the servo to GPIO 18. When analyzing our rain sensor, we collaborated to record the manual multimeter readings alongside the automated ADC samples to evaluate the experimental design.

Through these activities, I developed a deeper technical understanding of how embedded systems integrate code with physical components. I learned that a microcontroller combines a microprocessor core with memory, programmable I/O ports, and control logic on a single integrated circuit, distinguishing it from a standard general-purpose processor. In MicroPython, interfacing with hardware requires importing the machine module to initialize GPIO pin directions, such as setting an LED pin to Pin.OUT, and manipulating output states using the .value() function. I also gained a clear understanding of analog versus digital concepts, learning how an ADC converts continuous analog voltage signals into digital values for computational

analysis. Furthermore, I learned how PWM signals govern actuators. By adjusting the duty cycle, the average voltage is altered based on the equation $V_{avg} = D * V_H + (1 - D) * V_L$. Watching my partner wire breadboards also taught me the logic of electrical paths, reinforcing that current flows from the power supply through resistive loads to ground, and that components should never share the same row.

Overall, this module presented several troubleshooting challenges that required problem-solving skills, making the experience highly rewarding. A challenge we encountered was understanding the pin mapping of the Arduino Nano ESP32 board, which uses different physical pin labels compared to its internal ESP32 GPIO numbers. Another significant challenge was identifying flaws in our experimental data analysis. During our sensor testing, our automated ADC readings failed to detect any voltage change when the sensor was blown on, resulting in a standard deviation of zero where a change should have occurred. Additionally, having to estimate the fluctuating manual multimeter readings introduced human error into our calculations. While we chose not to use the recommended rubber ducky debugging method, we successfully navigated the bugs by repeatedly testing our code and consulting with the course staff. When our circuits finally functioned and we pushed our updated codes to our GitHub repository, it felt like a massive accomplishment.